Model_Application

In [1]:
```
import os
os.chdir('C:\\Users\\User\\Desktop\\Results')
print(os.getcwd())
print(os.listdir())
```

```
C:\Users\User\Desktop\Results
['.ipynb_checkpoints', '260512', 'checking smthn', 'Counts', 'Flourescent ovary stud
y deconvoluted', 'Flourescent ovary study red only tiled', 'Flourescent ovary study
red only tiled restored', 'Fluorescent ovary study', 'Fluorescent ovary study red on
ly', 'FM_4', 'from FOS folder', 'graticule 2x nikon 0.01m.tif', 'jan 2026 7 mth', 'l
abeling', 'Ovarian Reserve Measurements.xlsx', 'stack of overlays OG in Red restored
in Green.tif', 'Step_3_Model_Application.ipynb', 'Validation_with_Kates_images']
```

Import the packages needed for model application and configure the model to that of the model that was generated in training

In [2]:
```
from __future__ import print_function, unicode_literals, absolute_import, division
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
%config InlineBackend.figure_format = 'retina'

import tifffile
from tifffile import imread

from csbdeep.utils import Path, download_and_extract_zip_file, plot_some
from csbdeep.io import load_training_data, save_tiff_imagej_compatible
from csbdeep.models import CARE
```

In [3]:
```
from csbdeep.data.prepare import PercentileNormalizer
```

In [4]:
```
# Define the folder containing the input images
input_folder = "flourescent ovary study red only tiled"

# Define the folder to save the restored images
output_folder = "flourescent ovary study red only tiled restored"

# Create the output folder if it doesn't exist
os.makedirs(output_folder, exist_ok=True)
```

In [5]:
```
# Load the CARE model
model = CARE(config=None, name='FM_4')
```

```
Loading network weights from 'weights_best.h5'.
```

The following code will go through the input folder and look for each subdirectory that contains images.

In this case the input folder contained subdirectories for each ovary containing the 36 tiles of the red channel for that image.

This then applied the model to each of the 36 tiles and saved them to a corresponding subdirectory in the output folder for the restored image.

In [6]:
```
for root, dirs, files in os.walk(input_folder):
    # Extract the current subdirectory's name
    for directory in dirs:
        subdirectory_path = os.path.join(root, directory)

        # Create corresponding output directory
        output_subdirectory = os.path.join(output_folder, f"{directory}_restore
        if not os.path.exists(output_subdirectory):
            os.makedirs(output_subdirectory)

        # Process each file in the current subdirectory
        for filename in os.listdir(subdirectory_path):
            if filename.endswith(".tif"):
                # Construct the full path of the input image
                input_image_path = os.path.join(subdirectory_path, filename)

                # Load the input image
                x = imread(input_image_path)

                # Apply the model to restore the image
                restored = model.predict(x, axes='YX')

                # Construct the full path of the output image
                output_image_path = os.path.join(output_subdirectory, filename)

                # Save the restored image
                tifffile.imwrite(output_image_path, restored)

        print(f"Processing complete for subdirectory: {directory}")

print("Restoration complete for all subdirectories.")
```

```
Processing complete for subdirectory: 257455 Left Ovary In Situ 001 10s 3x_tiled
Processing complete for subdirectory: 257455 Left Ovary Side B 004 10s 4x_tiled
Restoration complete for all subdirectories.
```